

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине Построение и анализ алгоритмов
ТЕМА: «КНУТ-МОРРИС-ПРАТТ»

Студент гр. 4343

Миридаштаки Ф.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы

Изучить принцип работы алгоритма Кнута-Морриса-Пратта, реализовать его для решения задачи поиска вхождений заданного шаблона в текст, а также для проверки того, является ли одна строка циклическим сдвигом другой.

Задание 1

Реализуйте алгоритм КМП и с его помощью для заданных шаблона $P(|P| \leq 25000)$ и текста $T(|T| \leq 5.000.000)$ найдите все вхождения P в T .

Входные данные:

Первая строка – P

Вторая строка – T

Выходные данные:

Индексы начал вхождений P в T , разделенные запятой, если P не входит в T , то вывести -1.

Sample Input:

ab

abab

Sample Output:

0,2

Задание 2

Заданы две строки $A(|A| \leq 5.000.000)$ и $B(|B| \leq 5.000.000)$.

Определить, является ли A циклическим сдвигом B (это значит, что A и B имеют одинаковую длину и A состоит из суффикса B , склеенного с префиксом B). Например defabc является циклическим сдвигом abcdef.

Входные данные:

Первая строка – A

Вторая строка – B

Выходные данные:

Если A является циклическим сдвигом B, индекс начала строки B в A, иначе вывести -1. Если возможно несколько сдвигов вывести первый индекс.

Sample Input:

defabc

abcdef

Sample Output:

3

Выполнение работы

Описание алгоритма

Алгоритм Кнута-Морриса-Пратта представляет собой метод быстрого поиска подстроки внутри строки без лишних повторных сравнений.

В основе алгоритма лежит префикс-функция, которая для каждой позиции паттерна вычисляет длину наибольшего собственного префикса, одновременно являющегося суффиксом. Эти значения позволяют при несовпадении символов не возвращаться к началу паттерна, а перепрыгивать на нужную позицию.

При поиске вхождений используется один указатель на текущую позицию в тексте и один — на текущую позицию в паттерне. На каждом шаге символы сравниваются: при совпадении оба указателя продвигаются вперёд, а при полном прохождении паттерна фиксируется найденное вхождение. При несовпадении указатель паттерна сдвигается согласно значению префикс-функции, что позволяет продолжить поиск с оптимальной позиции.

Для определения циклического сдвига алгоритм применяется следующим образом: поиск строки В ведётся внутри виртуально удвоенной строки А, что гарантирует наличие любого возможного сдвига.

Оценка сложности

Временная сложность алгоритма является линейной — $O(n+m)$, где n — длина текста, m — длина паттерна. Построение префикс-функции занимает $O(m)$, обход текста — $O(n)$, так как каждый символ обрабатывается не более одного раза. Для задачи циклического сдвига сложность составляет $O(2n+m)$, поскольку текст обходится дважды виртуально.

Пространственная сложность равна $O(m)$, так как требуется хранить только массив значений префикс-функции размером с паттерн.

Код программы содержит реализацию следующих функций:

- *compute_prefix(p)* — вычисляет для каждой позиции паттерна длину наибольшего префикса, совпадающего с суффиксом.

- $kmp_search(p, t)$ — используя результаты префикс-функции, находит все позиции вхождения паттерна в текст.
- $kmp_first(p, b)$ — используя результаты префикс-функции, определяет, является ли одна строка циклическим сдвигом другой, и возвращает первый индекс.

Тестирование

Обе программы были протестированы на различных входных данных.

Соответственно составлены две таблицы:

Таблица 1. Тестирование задачи поиска шаблона.

Входные данные	Выходные данные
ab abab	0,2
abc abc	0
abcd abc	-1
aa aaaaa	0,1,2,3
abac ababacabac	2,6
a b	-1

Таблица 2. Тестирование задачи поиска циклического сдвига.

Входные данные	Выходные данные
defabc abcdef	3
abc abcd	-1
abc def	-1
a a	0

abcabcabc bcabcabca	7
aaaaabaa baaaaaaa	5
abcdef cdefab	2

```

farid@Farid-M1-Mac src % python3 kmp.py
ab
abca
Computing prefix function

pi[0] = 0 by definition

Characters at positions (0) and (1) are different
pi[1] = 0
i increases by 1

Prefix function values for each character of the pattern:
a b
0 0

Computing occurrences

Characters at text position (0) and pattern position (0) matched
k increases by 1

Characters at text position (1) and pattern position (1) matched
Full match found in interval {0:1}
k moves to 0

Characters at text position (2) and pattern position (0) are different
Pattern start did not match current text position. i increases by 1.

Characters at text position (3) and pattern position (0) matched
k increases by 1

Occurrences found:
0

```

Рисунок 1 – Результат работы программы kmp.py


```
farid@Farid-M1-Mac src % python3 shift.py
abc
bca
Computing prefix function

pi[0] = 0 by definition

Characters at positions (0) and (1) are different
pi[1] = 0
i increases by 1

Characters at positions (0) and (2) are different
pi[2] = 0
i increases by 1

Prefix function values for each character of the pattern:
b c a
0 0 0

Computing cyclic shift

Characters at positions (0) and (0) are different
j increases by 1

Characters at positions (0) and (1) matched
k increases by 1
j increases by 1

Characters at positions (1) and (2) matched
k increases by 1
j increases by 1

Characters at positions (2) and (0) – modulo length, matched
k increases by 1
j increases by 1

Full match found

Shift by 1
```

Рисунок 2 – Результат работы программы shift.py

Исследование

Исследуем эффективность алгоритма Кнута-Морриса-Пратта на различных объёмах входных данных. Измерения проводились на языке Python для разных размеров текста и паттерна.

Таблица 3. Исследование эффективности по времени.

Размер текста	Размер паттерна	Затраченное время (с)
1.000	1	0.000147
1.000	10	0.000154
100.000	1	0.012738
100.000	100	0.010861
100.000	10.000	0.011385
1.000.000	1	0.100852
1.000.000	1.000	0.10182
1.000.000	100.000	0.112454
5.000.000	1	0.509429
5.000.000	100	0.506219
5.000.000	10.000	0.510699
5.000.000	1.000.000	0.661234

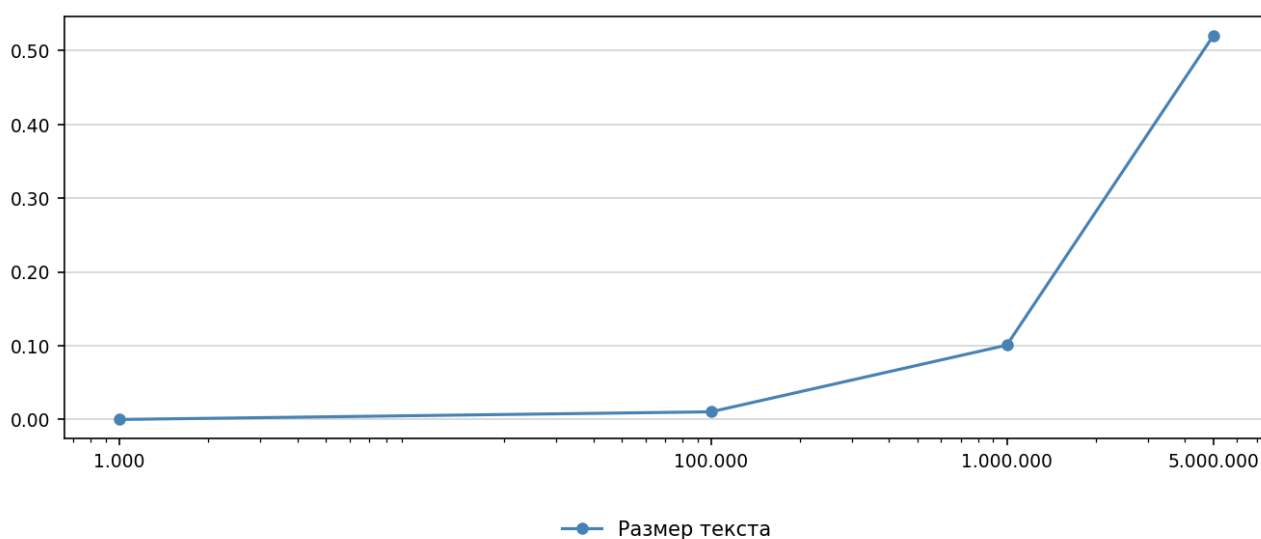


Рисунок 3 – График зависимости времени работы алгоритма от размера текста

Можно сделать следующие выводы по исследованию:

1. Время работы алгоритма прямо пропорционально размеру текста — при увеличении текста в 5000 раз время возрастает примерно в 3500 раз.
2. Время работы практически не зависит от размера паттерна, что подтверждает линейную сложность алгоритма $O(n+m)$.

Выводы

В ходе выполнения лабораторной работы был изучен алгоритм Кнута-Морриса-Пратта и принцип построения префикс-функции. Реализованы две задачи на языке Python: поиск всех вхождений заданного паттерна в текст, а также проверка того, является ли одна строка циклическим сдвигом другой.

ПРИЛОЖЕНИЕ

ИСХОДНЫЙ КОД ПРОГРАММЫ

Имя файла: kmp.py

```
import sys
input = sys.stdin.readline

def compute_prefix(p):
    m = len(p)
    pi = [0] * m
    k = 0
    print("Computing prefix function")
    print()
    print("pi[0] = 0 by definition")
    print()
    for i in range(1, m):
        while k > 0 and p[k] != p[i]:
            print("k > 0 and characters " + p[k] + "(" + str(k) + ")")
            print("and " + p[i] + "(" + str(i) + ") are different")
            print("k is replaced by previous prefix value at index")
            print("[k-1] = " + str(pi[k - 1]))
            print()
            k = pi[k - 1]
        if p[k] == p[i]:
            print("Characters at positions (" + str(k) + ") and (" +
            str(i) + ") matched")
            k += 1
            print("k increases by 1")
            pi[i] = k
            print("pi[" + str(i) + "] = " + str(k))
            print("i increases by 1")
            print()
        else:
            print("Characters at positions (" + str(k) + ") and (" +
            str(i) + ") are different")
            pi[i] = 0
            print("pi[" + str(i) + "] = 0")
            print("i increases by 1")
            print()
    return pi

def kmp_search(p, t):
    m, n = len(p), len(t)
    pi = compute_prefix(p)

    print("Prefix function values for each character of the")
    print("pattern:")
    print(" ".join(list(p)))
    print(" ".join(map(str, pi)))
    print()

    print("Computing occurrences")
    print()

    results = []
    k = 0
```

```

        for i in range(n):
            while k > 0 and p[k] != t[i]:
                print("k > 0 and characters " + t[i] + "(" + str(i) + ")
and " + p[k] + "(" + str(k) + ") are different")
                print("k is replaced by previous prefix value = " +
str(pi[k - 1]))
                print()
                k = pi[k - 1]
            if p[k] == t[i]:
                print("Characters at text position (" + str(i) + ") and
pattern position (" + str(k) + ") matched")
                k += 1
                if k == m:
                    start = i - m + 1
                    print("Full match found in interval {" + str(start)
+ ":" + str(i) + "}")
                    results.append(start)
                    k = pi[k - 1]
                    print("k moves to " + str(k))
                else:
                    print("k increases by 1")
            else:
                print("Characters at text position (" + str(i) + ") and
pattern position (" + str(k) + ") are different")
                if k == 0:
                    print("Pattern start did not match current text
position. i increases by 1.")
                print()
            return results

p = input().strip()
t = input().strip()

res = kmp_search(p, t)

if res:
    print("Occurrences found:")
    print(','.join(map(str, res)))
else:
    print(-1)

```

Имя файла: shift.py

```

import sys
input = sys.stdin.readline

def compute_prefix(p):
    m = len(p)
    pi = [0] * m
    k = 0
    print("Computing prefix function")
    print()
    print("pi[0] = 0 by definition")
    print()
    for i in range(1, m):
        while k > 0 and p[k] != p[i]:
            print("k > 0 and characters " + p[k] + "(" + str(k) + ") and "
+ p[i] + "(" + str(i) + ") are different")
            print("k is replaced by previous prefix value at index [k-1] =

```

```

" + str(pi[k - 1]))
    print()
    k = pi[k - 1]
    if p[k] == p[i]:
        print("Characters at positions (" + str(k) + ") and (" + str(i)
+ ") matched")
        k += 1
        print("k increases by 1")
        pi[i] = k
        print("pi[" + str(i) + "] = " + str(k))
        print("i increases by 1")
        print()
    else:
        print("Characters at positions (" + str(k) + ") and (" + str(i)
+ ") are different")
        pi[i] = 0
        print("pi[" + str(i) + "] = 0")
        print("i increases by 1")
        print()
    return pi

def kmp_first(p, b):
    m = len(p)
    n = len(b)
    pi = compute_prefix(p)

    print("Prefix function values for each character of the pattern:")
    print(" ".join(list(pi)))
    print(" ".join(map(str, pi)))
    print()

    print("Computing cyclic shift")
    print()

    k = 0
    for j in range(2 * n):
        c = b[j % n]
        while k > 0 and p[k] != c:
            print("k > 0 and characters " + p[k] + "(" + str(k) + ") and "
+ c + "(" + str(j) + ") are different")
            print("k is replaced by previous prefix value = " + str(pi[k -
1]))

            print()
            k = pi[k - 1]
        if p[k] == c:
            if j >= n:
                print("Characters at positions (" + str(k) + ") and (" +
str(j % n) + ") - modulo length, matched")
            else:
                print("Characters at positions (" + str(k) + ") and (" +
str(j % n) + ") matched")
            k += 1
            print("k increases by 1")
            print("j increases by 1")
            print()
            if k == m:
                pos = j - m + 1
                if pos < n:
                    print("Full match found")
                    print()
                    return pos
                k = pi[k - 1]
        else:
            print("Characters at positions (" + str(k) + ") and (" + str(j %

```

```

n) + ") are different")
        print("j increases by 1")
        print()
        return -1

a = input().strip()
b = input().strip()

if len(a) != len(b):
    print(-1)
else:
    res = kmp_first(b, a)
    if res == -1:
        print(-1)
        print("A is not a cyclic shift of B")
    else:
        print("Shift by " + str(res))

```